Cybersecurity in Autonomous Vehicles

MODULE 5.2

Agenda

- How Autonomous systems work?
- Why Autonomous Vehicles (AV) security matters
- Threat Landscape
- CAN Bus & IDS
- Sensor Spoofing
- Putting It All Together

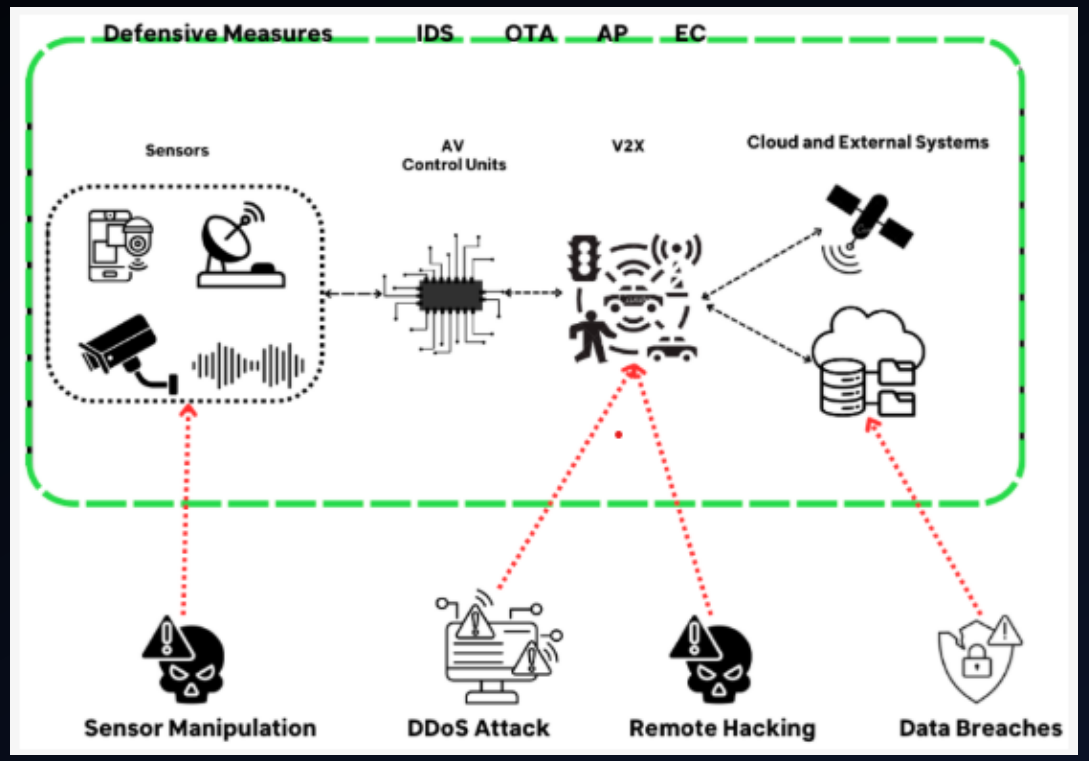
How Autonomous Systems Work?

- **Perception:** Sensors (Camera, LiDAR, RADAR, GPS) observe the environment. The vehicle builds a real-time picture of the world around it.
- **Prediction:** ML models interpret sensor data to classify objects, detect pedestrians, read traffic signs, and anticipate what other vehicles will do.
- **Planning:** The system decides the safest path, speed, and maneuver based on predicted states of the environment.
- **Action:** Commands are sent over the CAN bus to actuators — brakes, throttle, steering — to execute the plan.

Why Autonomous Vehicles (AV) security matters?

- Software directly controls steering, braking and acceleration
- Large attack surface: sensors, ECUs, V2X, cloud
- Cyberattacks can cause real-world physical harm
- Breaches damage regulatory compliance and public trust

The AV attack surface



Attackers can target every layer of an AV

- **Physical / Sensor Layer:** GPS Spoofing, LiDAR laser injection, camera blinding, ultrasound jamming
- **In-Vehicle Network (CAN Bus):** DoS Flooding, message spoofing, replay attacks, fuzzy frame injection
- **Perception:** Adversarial patches on traffic sign boards, FGSM image perturbations, model poisoning
- Communication layer: V2X replay attacks, man-in-the-middle on OTA updates

The CAN Bus: Vehicle’s Internal Nervous System

CAN Frame Structure

Timestamp	CAN_ID	DLC	DATA0-DATA7	Flag – 0,1,2

Why CAN is Insecure by design?

- **No authentication:** any ECU can send any message; no way to verify the sender
- **No encryption:** all data is transmitted as plain text, visible to every node
- **Broadcast by design:** every message seen by all ECUs simultaneously
- **Priority arbitration:** low CAN ID = higher priority; attackers flood with high-priority frames

Common CAN Attack Types

- **Denial of Service (DoS) :** Flood bus to block legitimate ECUs
- **Fuzzy:** sends random frames to destabilize ECUs
- **Spoofing:** Inject fake messages as trusted ECU
- **Replay:** capture and retransmit real packets at wrong time

Intrusion Detection System (IDS)

Machine Learning based IDS in This Workshop

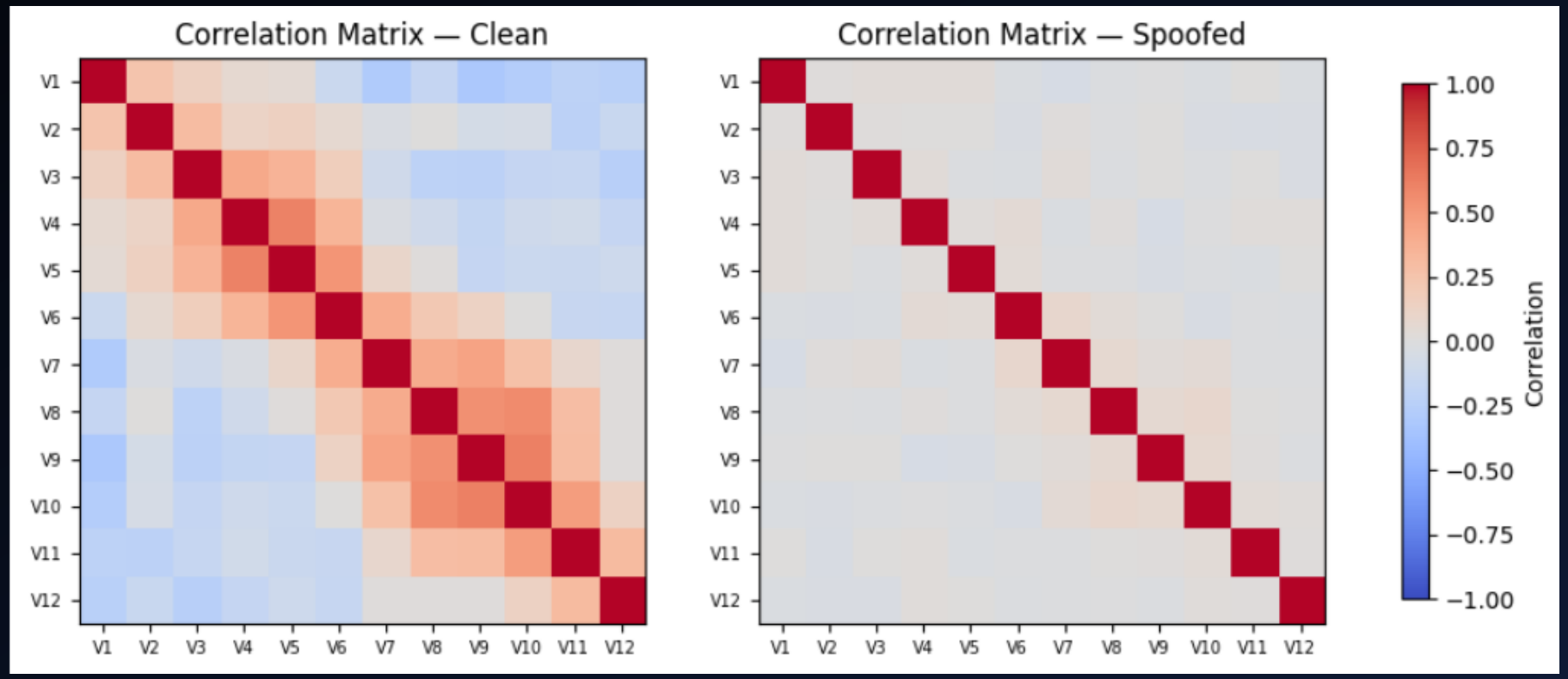
- Model: Any classification model can be applied on labeled CAN data
- Approach: Using machine learning models from scikit-learn
  - Load, clean and pre-process Raw CAN data files
  - Perform feature engineering
  - Model Training and Evaluation
- Key Metrics for IDS Evaluation:
  - Precision: to identify false alarms
  - Recall: to identify missed attacks
  - ROC-AUC: to obtain ranking between different attack types

Sensor Spoofing

Sensor Spoofing — Attacking the Physical Layer

Spoofing attacks inject false data into sensor feed before they reach the model

Formula: $x_spoofed = x_clean + N(0, \sigma^2)$



Spoofing Types and Risks

Type	Attack Strategy	Risk
LiDAR Spoofing	Laser pulses → phantom obstacles or blind spots	Fake objects; missing real obstacles
Camera Spoofing	Glare / IR blinding / adversarial projection	Object misclassification; lane loss
GPS/GNSS Spoofing	Broadcast fake signals → shift position	Vehicle navigates wrong route
RADAR Spoofing	Jam or replay signals → false velocity	Wrong speed estimates

Sensor Spoofing

Sensor Spoofing in This Workshop

- Model: Any classification model can be applied on single-sensor or fused multi-sensor data
- Approach:
  - Load, clean and pre-process sensors dataset
  - Apply gaussian noise spoofing to any one sensor at inference time
  - Model Training and Evaluation
- Key Metrics for IDS Evaluation:
  - Attack Strength (σ): controls how much noise is injected
  - Clean accuracy: Baseline model performance when sensors are normal
  - Spoofed accuracy: To measure the resilience of the model when input is corrupted

Putting It All Together

Next Steps & Q&A

- Questions on any of the attack techniques?
- Workflow:
  - Start with Notebook 1 → 2 → 3
- Encourage:
  - Experimenting with hyperparameters
  - Trying combinations of techniques
  - Thinking about your own deployment scenarios